

Why this one prediction?

Global importance (deck 2) says temp and yr matter *overall*. But the model predicts **this particular day** at **4311 rentals** (below the ~ 4500 average) — which features drove *that*?

Global tools can't answer per-prediction. **SHAP** can: it explains a **single prediction**, and the same idea rolls up into global plots.

Still the bike-sharing random forest from decks 1–2.

Outline

What SHAP is

The SHAP plots (how to read them)

SHAP in practice

LIME

Counterfactual explanations

What SHAP is

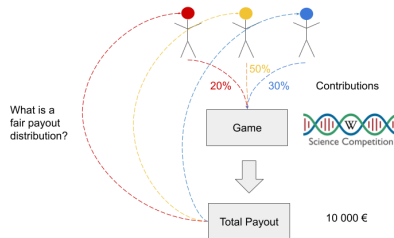
A fair split of the prediction among the features.

The idea: a fair payout

Treat the prediction as a **payout to split fairly** among the features — like a team bonus among players who contributed differently.

SHAP gives each feature its **fair share**: its contribution *averaged over all the orders* in which features could be added to the model.

That averaging is the whole game-theory part.
Next: how that fair share is *actually computed* — then we read the plots.



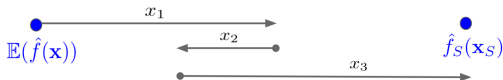
Source: LMU IML (CC BY 4.0).

FROM GAME THEORY TO MACHINE LEARNING

- Game: Make prediction $\hat{f}(x_1, x_2, \dots, x_p)$ for a single observation $\mathbf{x}$
- Players: Features $x_j, j \in \{1, \dots, p\}$ which cooperate to produce a prediction
 - $\rightsquigarrow$ How can we make a prediction with a subset of features without changing the model?
 - $\rightsquigarrow$ PD function: $\hat{f}_S(\mathbf{x}_S) := \int_{X_{-S}} \hat{f}(\mathbf{x}_S, X_{-S}) d\mathbb{P}_{X_{-S}}$ ("removing" by marginalizing over $-S$)
- Value function / payout of coalition $S \subseteq P$ for observation $\mathbf{x}$:

$$v(S) = \hat{f}_S(\mathbf{x}_S) - \mathbb{E}_{\mathbf{x}}(\hat{f}(\mathbf{x})), \text{ where } \hat{f}_S : \mathcal{X}_S \mapsto \mathcal{Y}$$

$\rightsquigarrow$ subtraction of $\mathbb{E}_{\mathbf{x}}(\hat{f}(\mathbf{x}))$ ensures that v is a value function with $v(\emptyset) = 0$



- Marginal contribution: $v(S \cup \{j\}) - v(S) = \hat{f}_{S \cup \{j\}}(\mathbf{x}_{S \cup \{j\}}) - \hat{f}_S(\mathbf{x}_S)$
 - $\rightsquigarrow \mathbb{E}_{\mathbf{x}}(\hat{f}(\mathbf{x}))$ cancels out due to the subtraction of value functions

SHAPLEY VALUE - DEFINITION

► Shapley (1953)

► Strumbelj et al. (2014)

Shapley value ϕ_j of feature j for observation $\mathbf{x}$ via **order definition**:

$$\phi_j(\mathbf{x}) = \frac{1}{|P|!} \sum_{\tau \in \Pi} \underbrace{\hat{f}_{S_j^\tau \cup \{j\}}(\mathbf{x}_{S_j^\tau \cup \{j\}}) - \hat{f}_{S_j^\tau}(\mathbf{x}_{S_j^\tau})}_{\text{marginal contribution of feature } j}$$

- Interpretation: Feature x_j contributed ϕ_j to difference between $\hat{f}(\mathbf{x})$ and average prediction
 $\rightsquigarrow$ Note: Marginal contributions and Shapley values can be negative
- For exact computation of $\phi_j(\mathbf{x})$, the PD function $\hat{f}_S(\mathbf{x}_S) = \frac{1}{n} \sum_{i=1}^n \hat{f}(\mathbf{x}_S, \mathbf{x}_{-S}^{(i)})$ for any set of features S can be used which yields

$$\phi_j(\mathbf{x}) = \frac{1}{|P|! \cdot n} \sum_{\tau \in \Pi} \sum_{i=1}^n \hat{f}(\mathbf{x}_{S_j^\tau \cup \{j\}}, \mathbf{x}_{-\{S_j^\tau \cup \{j\}\}}^{(i)}) - \hat{f}(\mathbf{x}_{S_j^\tau}, \mathbf{x}_{-S_j^\tau}^{(i)})$$

$\rightsquigarrow$ Note: $\hat{f}_S$ marginalizes over all other features $-S$ using all observations $i = 1, \dots, n$

By hand: three friends split the payout

Adi, Budi and **Gogi** run a stall together.

Working in different groups, they would earn different amounts — the group's **payout** $v(S)$:

Together they earn **90**. **What is each one's fair share** of that 90?

Names: qarakusi.am

who works together S	$v(S)$
nobody	0
Adi	10
Budi	20
Gogi	30
Adi + Budi	40
Adi + Gogi	50
Budi + Gogi	60
all three	90

Fair share = average marginal contribution

Adi's extra contribution depends on who is **already there**:

who's before Adi	Δ_{Adi}	#orders
nobody (Adi first)	$10 - 0 = 10$	2
Budi	$40 - 20 = 20$	1
Gogi	$50 - 30 = 20$	1
Budi & Gogi	$90 - 60 = 30$	2

Average over all **6 orders**:

$$\phi_{\text{Adi}} = \frac{2(10) + 20 + 20 + 2(30)}{6} = \mathbf{20}.$$

Same for each friend:

$$\phi_{\text{Adi}}=20, \quad \phi_{\text{Budi}}=30, \quad \phi_{\text{Gogi}}=40$$

They **sum to 90** — the whole payout is split, nothing lost or invented (*efficiency*).

In a model: the players are **features**, the payout is $\hat{f}(\mathbf{x}) - \mathbb{E}[\hat{f}]$, and each fair share is that feature's **SHAP value** ϕ_j .

One prediction = base value + the SHAP values

Every SHAP explanation is one simple sum:

$$\underbrace{\hat{f}(\mathbf{x})}_{\text{this prediction}} = \underbrace{\mathbb{E}[\hat{f}(X)]}_{\text{base value (avg. prediction)}} + \sum_j \phi_j,$$

where ϕ_j is feature j 's SHAP value.

- ▶ $\phi_j > 0$ pushes this prediction **up**; $\phi_j < 0$ pushes it **down**.
- ▶ **Every SHAP plot visualizes this sum** — for one row, or aggregated over many.

Exact SHAP explodes — TreeSHAP & KernelSHAP to the rescue

3 features $\rightarrow 3! = 6$ orders ($2^3 = 8$ coalitions): easy by hand. But it **explodes** with the number of features:

features	orders $n!$
3	6
10	3.6 million
20	2.4×10^{18}

You **cannot** enumerate them all $\Rightarrow$ we **approximate**.

KernelSHAP: *sample* a handful of coalitions and fit a **weighted linear model** to recover the ϕ_j . Works on **any** model — but approximate and slow.

TreeSHAP: exploits the **tree structure** to get **exact** SHAP in *polynomial* time — fast + exact for trees/boosting $\Rightarrow$ the **default**.

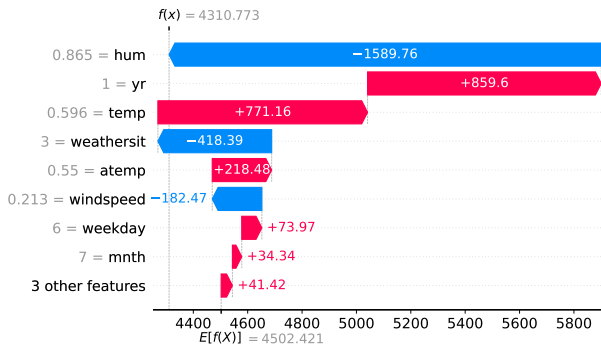
The SHAP plots — how to read them

One local plot, three global ones. Same SHAP values, different views.

Waterfall: one prediction, fully explained

Start at the **base value** (~ 4502 rentals) and add each feature's SHAP value until you reach **this day's prediction** (4311).

Read it: yr (+860) and temp (+771) push it **up**, but a very humid day (hum -1590) and poor weathersit (-418) push it **down** — landing just below average.

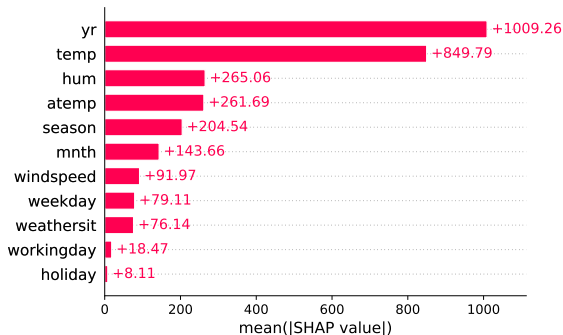


Bar: global importance (mean |SHAP|)

Average the **magnitude** of each feature's SHAP over all rows:

$$\text{importance}_j = \frac{1}{n} \sum_i |\phi_j^{(i)}|$$

- ▶ One bar per feature — **reads like the PFI bar** from deck 2.
- ▶ yr, temp dominate — roughly agreeing with PFI.



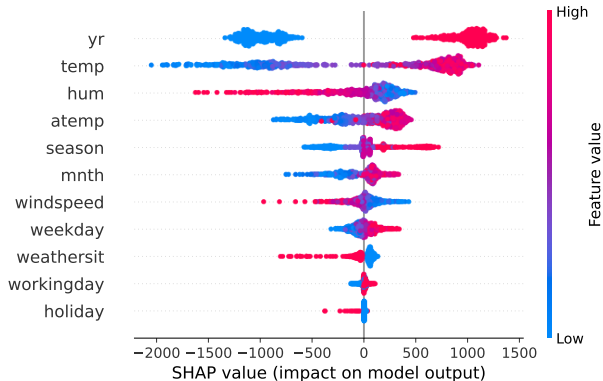
Beeswarm: the signature SHAP plot

One dot per row, per feature.

x = SHAP value; **colour** = feature value.

Read it:

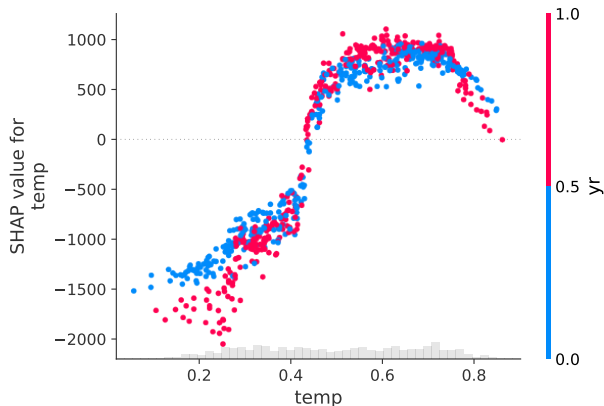
- **Spread** = how much the feature matters.
- **Colour pattern** = direction: high temp (red) → positive SHAP (more rentals); low (blue) → negative.



Dependence: the effect shape (& interactions)

SHAP value of a feature **vs. its value** — one dot per row.

- Shows the **effect shape**: rentals **rise sharply** with temp, peak, then **dip at extreme heat** ($\approx$ a PDP).
- Colour by a second feature (here yr) reveals **interactions**.
- Per-instance — *like ICE did for PDP*, it shows the spread the average hides.



How SHAP ties the methods together

- ▶ **TreeSHAP** (previous slide) makes SHAP the **default** for trees/boosting.
- ▶ **SHAP unifies what we've seen:**
 - ▶ the **bar** $\approx$ PFI (global importance),
 - ▶ the **dependence** plot $\approx$ PDP (effect shape),
 - ▶ **plus** a *local* view (the waterfall) the others can't give — with a consistency guarantee (use a feature more $\Rightarrow$ its SHAP can't fall).

But like every method here, correlation muddies SHAP too — the temp/atemp credit split stays ambiguous. SHAP also explains the model's *score*: if that score isn't **calibrated** (L13), SHAP faithfully explains a *miscalibrated* number. Still **model behaviour, not causation** (deck 2's ice cream), and rich plots are **easy to over-read**.

SHAP in code

```
import shap
explainer = shap.TreeExplainer(model)           # fast & exact for trees/  
                                                boosting  
sv = explainer(X)                               # SHAP values for every  
                                                row  
  
shap.plots.waterfall(sv[i])                    # one prediction  
shap.plots.bar(sv)                             # global: mean |SHAP|  
shap.plots.beeswarm(sv)                        # global: dot per row,  
                                                colour = value  
shap.plots.scatter(sv[:, "temp"])              # dependence: effect  
                                                shape
```

shap.Explainer(model, X) auto-picks the right method (TreeSHAP here).

LIME: the lighter local cousin

A simple, model-agnostic local explanation — with sharper edges.

LIME: fit a simple model, locally

LIME explains one prediction with a **local surrogate**: jitter the inputs around the instance, weight by **closeness**, and fit a **simple sparse linear model** — its weights say “these few features pushed this prediction.” Works on tabular, text, & images.

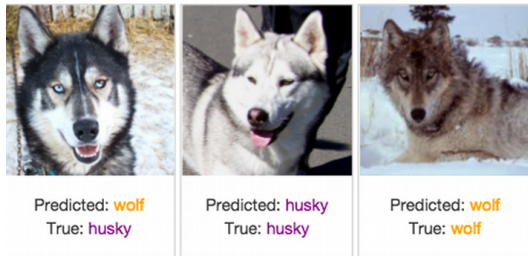
Predict-first: a husky-vs-wolf classifier is ~90% accurate. What did LIME reveal it actually keys on?

LIME: fit a simple model, locally

LIME explains one prediction with a **local surrogate**: jitter the inputs around the instance, weight by **closeness**, and fit a **simple sparse linear model** — its weights say “these few features pushed this prediction.” Works on tabular, text, & images.

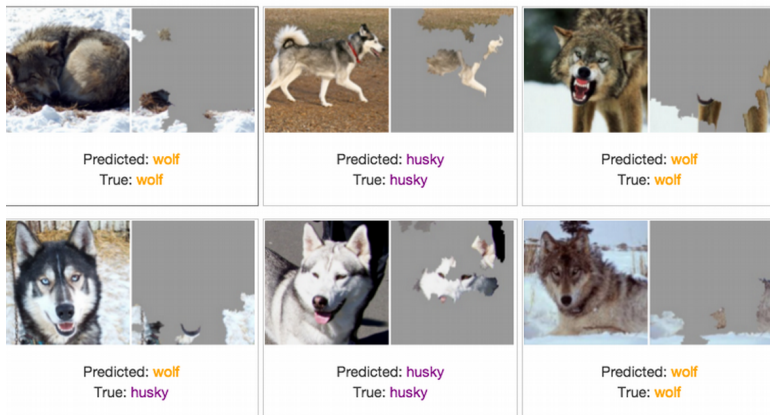
Predict-first: a husky-vs-wolf classifier is ~90% accurate. What did LIME reveal it actually keys on?

The snow in the background — not the animal. A *clever-Hans* model (right answer, wrong reason), caught by a local explanation.



Husky vs. wolf: LIME highlights snow pixels. LMU IML (CC BY 4.0).

LIME's proof: the explanation *is* the snow



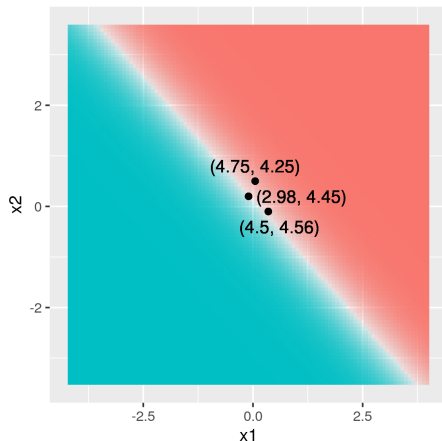
In each pair: the photo (left) and **LIME's explanation** (right — the kept superpixels). The model's "reason" is the **snow**, not the animal, so the snowy **husky** (bottom-left) is misread as a **wolf**. LMU IML (CC BY 4.0) / Ribeiro et al. 2016.

LIME caveats, and SHAP vs. LIME

Caveats

- ▶ **Unstable**: re-run and the explanation can change.
- ▶ The **neighbourhood** (how far to jitter, how to weight) is an *arbitrary* choice that moves the answer.

SHAP vs. LIME: SHAP has theory (consistency) + richer plots, but is slower; LIME is simpler/faster but less stable. **SHAP is the modern default.**



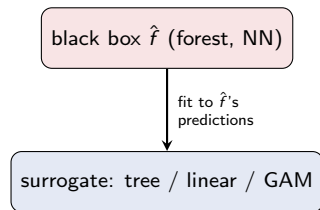
Repeated LIME fits near one point **disagree** — unstable. LMU IML.

LIME's global sibling: the surrogate model

LIME fits a simple model *locally*, around one prediction. Its **global** counterpart does the same for the *whole* model:

Global surrogate: train an interpretable model — a shallow tree, a linear model, a GAM — to **mimic the black box's predictions** everywhere, then read the surrogate instead of the black box.

- ▶ Check its **fidelity** first (R^2 of surrogate vs. black-box output): a poor mimic is a misleading explanation.
- ▶ In effect it **distills the forest back into deck 1's glass box**.



read this instead

Counterfactual explanations

Not *why* the model decided — but *what would change* the outcome.

The smallest change that flips the prediction

SHAP and LIME answer “*why* this prediction?” A **counterfactual** answers a different, often more useful question:

What is the smallest change to the input that would flip the prediction?

- ▶ “Flip” needs a **decision**, so we switch from the (regression) bike model to the **ch. 3 bank-marketing** classifier — *will this client subscribe?* (realistic model, the leaky duration feature removed).
- ▶ A counterfactual is **actionable**: it tells you *what to change*, not just what mattered.

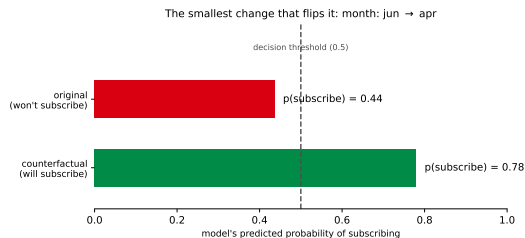
Example: a client predicted **won't subscribe** — the counterfactual says that *had the bank called in a different month*, the model would flip to **will subscribe**.

What makes a good counterfactual?

Five things we want at once:

- ▶ **Validity** — it actually flips the prediction.
- ▶ **Proximity** — close to the original (a small change).
- ▶ **Sparsity** — few features changed (easy to act on, easy to explain).
- ▶ **Plausibility** — stays in realistic data, no impossible rows (the extrapolation trap, deck 2).
- ▶ **Actionability** — change only what the user controls (raise a balance, not lower an age; never a protected attribute). The *recourse* angle.

Usually **several** valid counterfactuals exist — prefer the sparse, actionable ones.



Real DiCE counterfactual (bank): age, job, balance, outcome **frozen** (actionability); only the **call month** moved — one change, and the model crosses the decision line.

Counterfactuals in code

```
import dice_ml
d    = dice_ml.Data(dataframe=df, continuous_features=num,
                    outcome_name="y")
m    = dice_ml.Model(model=clf, backend="sklearn")    # any fitted
                    classifier
exp  = dice_ml.Dice(d, m, method="random")

cf = exp.generate_counterfactuals(
    client, total_CFs=3, desired_class="opposite",
    features_to_vary=["contact", "month", "campaign"])  #
                    actionable only
cf.visualize_as_dataframe()
```

Libraries: **DiCE** (diverse counterfactuals), **Alibi**. Constrain `features_to_vary` so the advice stays *actionable*.

Same data, four rankings

One bike dataset, (mostly) one model — four “importance” methods, four different top-3s:

Method	Top 3 features
Linear coefficients (deck 1)	yr, atemp, season
Tree impurity (deck 1)	temp, yr, atemp
Permutation / PFI (deck 2)	yr, atemp, temp
SHAP (deck 3)	yr, temp, hum

They mostly agree on yr/temperature, but disagree on the rest (season? hum? temp vs atemp?). **That** is what “importance is method-dependent” means — always say *which* importance, on *what* data.

Which method when? (the whole chapter)

You want. . .	Reach for. . .
to read the model itself	a glass-box model — coefficients / a tree / rules (deck 1)
global “which features matter”	PFI or SHAP bar (decks 2–3)
“how does a feature act”	PDP / ICE or SHAP dependence (decks 2–3)
“why this one prediction”	SHAP waterfall or LIME (deck 3)
“what would change the outcome”	counterfactuals — DiCE / Alibi (deck 3)

Whatever you pick: **(1)** correlated features mislead them all, **(2)** different methods rank differently (previous slide), **(3)** none of it is *causation*.

Recap

- ▶ **SHAP** fairly splits a prediction among features: $\hat{f}(\mathbf{x}) = \text{base} + \sum_j \phi_j$.
- ▶ Read the four plots: **waterfall** (one prediction), **bar** (global importance), **beeswarm** (global, colour = value), **dependence** (effect shape + interaction).
- ▶ **TreeSHAP** makes it the default for trees/boosting; SHAP **unifies** PFI & PDP and adds a local view.
- ▶ **LIME** is the simpler local cousin — handy but unstable.
- ▶ **Counterfactuals** answer “what would change the outcome” — the smallest **valid, sparse, actionable** edit that flips the prediction (DiCE).
- ▶ Everything explains the **model**, not causation — and plots are easy to over-read.

That's interpretability: read the model when you can, explain it when you can't — importance, effects, and per-prediction attributions, each answering a different question.